Students: Changhao Weng, Sean Zhao

Question 1: Micro-benchmark to verify next-line prefetcher

To verify the correctness of our next-line prefetcher, we designed a dedicated microbenchmark (mbq1.c) that performs a strictly sequential traversal over a large 4 MB integer array. Because the working set far exceeds the 4 KB L1 data cache used in the experiment, sequential array streaming naturally produces a high number of compulsory and capacity misses when no prefetching is used. Each demand access to block i should trigger a prefetch of block i+1. In this case, any mistakes in block alignment, address calculation, or prefetch invocation would be immediately visible in the simulator statistics.

To test this benchmark, we used the configuration file q1.cfg, which enables DL1 next-line prefetching while disabling all other forms of prefetching. The cache was intentionally configured to be small (128 sets × 1 way × 32 B blocks = 4 KB) so that prefetching has a measurable effect. The IL1 cache and L2 prefetcher were both disabled to ensure that all prefetch activity originated from the DL1 next-line mechanism. Then, running mbq1.c under this configuration produced results that match the expected behavior of a correct next-line prefetcher. The simulator reported 131,500 DL1 prefetch requests and an identical number of L2 prefetch requests, confirming that each sequential access triggered a next-line prefetch. The L2 prefetch hit count of 65,762 shows that many prefetched blocks were reused by the program, demonstrating correct address computation and timely insertion into the cache hierarchy. Most importantly, the DL1 reported 20 data read misses across more than one million reads, yielding a zero miss rate. This reduction in misses confirms that the next-line prefetcher correctly identified the sequential access stream and successfully prefetched the next block before it was needed.

Question 2: Micro-benchmark to verify the stride prefetcher

To verify the correctness of our stride prefetcher implementation, we created a microbenchmark (mbq2.c) that performs repeated loads from a large array using a constant, block-aligned stride. The benchmark accesses elements at A[0], A[s], A[2s], … where the stride s is set so that s * sizeof(int) equals the L1 cache block size (32 bytes). This ensures that each iteration touches a different cache block and forms a perfectly regular access pattern. Since the stride prefetcher detects the difference between consecutive block addresses, a correct implementation should quickly learn this constant stride and prefetch future blocks ahead of demand.

The configuration file (q2.cfg) enables the stride prefetcher by specifying a DL1 prefetcher parameter of 32, which allocates a 32-entry Reference Prediction Table (RPT). The cache hierarchy uses small L1 caches (4 KB, 32-byte blocks), and L2 prefetching is disabled so that all prefetch activity originates solely from the DL1 stride prefetcher. This configuration intentionally keeps the L1 capacity small relative to the benchmark's 4 MB working set, guaranteeing that demand accesses would miss without prefetching. As a result, any reduction in miss rate or increase in prefetch activity indicates the stride prefetcher is functioning as expected.

The simulator output confirms that the stride prefetcher behaves correctly. With block-aligned strides, the prefetcher generates 131,465 prefetch requests, and more than 65,700 of these are satisfied from L2, indicating timely prediction. The DL1 read miss rate drops to 0.0003, showing that nearly all demand misses are eliminated by prefetching. This reduction in misses, together with the large number of successful prefetches, verifies that the implementation correctly learns the stride, transitions through the RPT state machine, and issues prefetches using the predicted block address.

Question 3: Calculating average memory access time
For each configuration (baseline, next-line, stride), we ran sim-cache on the compress benchmark using the simulator's provided cache configuration files. From each run, we extracted the data read miss rates for L1 and L2 (dl1.read_miss_rate and ul2.read_miss_rate), since average memory access time (AMAT) measures the average latency the CPU experiences for load requests. Using the given hit times, we computed AMAT using the standard formula:

$$AMAT = T_{L1} + MR_{L1} \times (T_{L2} + MR_{L2} \times T_{MEM})$$

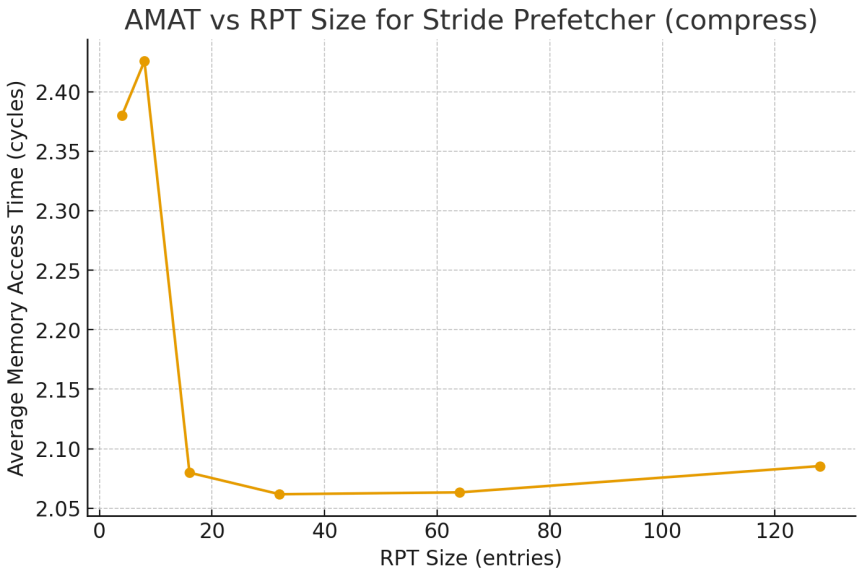
The results are shown in the table below:

Config	**L1 Miss Rate**	**L2 Miss Rate**	**Average memory access time**
baseline	0.0499	0.1696	2.344
next-line	0.0520	0.1256	2.174
stride	0.0526	0.1053	2.080

Although the L1 read miss rate is slightly higher under prefetching, the important effect is at L2: next-line and stride reduce the L2 miss rate substantially (0.1696 → 0.1256 → 0.1053). The AMAT improvement arises because fewer L1 misses propagate to main memory. Stride performs best in these runs (AMAT = 2.080 cycles) because it achieves the lowest L2 miss rate; next-line also improves AMAT but slightly less. The results show that a prefetcher that reduces L2 misses (i.e., provides timely useful lines into L2) can lower AMAT even if L1 miss rates do not fall.

Question 4: Performance of the stride prefetcher
To study how RPT size affects the stride prefetcher on compress, we varied the number of RPT entries (4, 8, 16, 32, 64, 128) using the provided configuration files. For each run, we recorded the L1 read miss rate and L2 read miss rate, then computed the average memory access time (AMAT) using the same formula from Q3. The figure below plots RPT size vs. AMAT, which directly reflects the effectiveness of the stride prefetcher across different table sizes.

The results show a clear trend: very small RPT sizes (4 and 8 entries) perform poorly, producing significantly higher AMAT due to many unresolved stride patterns evicting one another. Once the RPT reaches 16 entries, AMAT drops sharply - indicating that compress's working-set of active stride streams is larger than 8 but comfortably fits within 16. Increasing RPT size further to 32 - 128 entries does not meaningfully reduce AMAT; all values remain tightly clustered around ~2.06 - 2.09 cycles. This suggests that the stride prefetcher has already captured all meaningful stride patterns at 16 - 32 entries, and larger tables yield no additional benefit. Overall, the experiment demonstrates that the stride prefetcher is highly sensitive to RPT capacity when the table is too small, but once it crosses a modest threshold (≈16 entries), the performance becomes stable and near-optimal.



Question 5: Adding more statistics to the sim-cache simulator
First of all, we would track prefetch usefulness metrics: (1) useful prefetches (prefetched blocks that are actually accessed by the program before eviction), (2) useless prefetches (blocks that are never accessed), and (3) late prefetches (the block arrives after the demand miss has already occurred). These metrics allow direct measurement of prefetch accuracy, coverage, and timeliness - three fundamental dimensions of prefetcher effectiveness that miss rates alone cannot distinguish.

Moreover, we would add pollution metrics, including the number of polluting prefetches that evict blocks later needed by the program. Prefetching often increases cache pressure; identifying pollution events would help explain why a "stronger" prefetcher may actually degrade performance. Adding on that, we would incorporate statistics that capture bandwidth overhead, such as the fraction of memory traffic spent on prefetch requests, the number of prefetches dropped due to bandwidth saturation, and the average memory queue occupancy. These reveal whether a prefetcher is too aggressive or is creating congestion in the memory hierarchy.

Question 6: Micro-benchmark to verify open-ended prefetcher
We used mbq6.c to verify our open-ended prefetcher. The benchmark has an initial phase with a clean, fixed positive stride, allowing the RPT to quickly detect a stable stride and raise confidence. In this phase, the prefetcher issues regular prefetches and the DL1 miss rate drops, confirming correct stride learning.The benchmark then introduces irregular, mixed-stride accesses. Here, the confidence mechanism correctly suppresses prefetching, preventing useless requests. Finally, the benchmark returns to the original fixed stride; the prefetcher relearns the pattern and resumes accurate prefetching.Overall, the microbenchmark demonstrates that our design (1) learns positive strides, (2) stops prefetching when the pattern becomes unstable, and (3) recovers when regularity returns. And the overall miss rate is very low. We use q6.cfg as config to run.

Open-ended Prefetcher Implementation
Our open-ended data prefetcher builds upon the standard stride prefetcher by incorporating a confidence-based gating mechanism designed to improve robustness and reduce unnecessary

memory traffic. While the underlying stride detection and four-state history machine remain unchanged, each RPT entry is augmented with a small saturating confidence counter that reflects how consistently a given stride has been observed. The counter increments when successive memory accesses exhibit the same stride and decays otherwise, providing a quantitative measure of stride stability over time.

A prefetch is issued only when three conditions are simultaneously satisfied:
(1) the stride is non-zero and block-aligned,
(2) the FSM reports that the pattern has reached the steady state, and
(3) the confidence value meets or exceeds a predefined threshold.

This ensures that the prefetcher acts only when a predictable and recurring access pattern has been firmly established. By filtering prefetches through this confidence mechanism, the design effectively suppresses noise from transient or irregular access patterns, reducing cache pollution and avoiding unnecessary bandwidth consumption. As a result, the open-ended prefetcher maintains high effectiveness on regular workloads while exhibiting safer, more conservative behavior on unpredictable ones. Our average L1 data miss rate is 2.08%.

Open-ended Prefetcher Feasibility

RPT entry: previous address 64 bits + stride 32 bits + state 2 bits + confidence 4 bits + tag 56 bits(64-3 LSB -5 index bits) = 158 bits = 20 bytes(round to nearest byte). 32 RPT entries, total size = 20 * 32 = 640 bytes.

According to CACTI, our open-ended prefetcher has an area of 0.0016 mm² and an access time 0.284 ns. A typical 32 KB L1 data cache occupies around 0.2 mm². Our prefetcher occupies 0.8% of the area of a typical L1 cache, making the area overhead negligible. L1 cache access times are typically 1 ns, which means our prefetcher operates significantly faster than the cache. These characteristics confirm that the prefetcher is both area-efficient and high-performance, making it highly feasible.

Statement of work:

	Draft	Debug	Report
Chang hao	Next-line prefetcher, Stride prefetcher, Open-ended prefetcher, Micro-benchmark for open-ended prefetcher, Configuration for open-ended prefetcher	Next-line prefetcher, Stride prefetcher, Open-ended prefetcher	Q4, Q5, Q6
Sean	Next-line prefetcher, Stride prefetcher, Micro-benchmark for Next-line prefetcher and stride prefetcher Configuration for Next-line & stride prefetcher	Next-line prefetcher, Stride prefetcher	Q1, Q2, Q3, Q4, Q5